Assignement 1

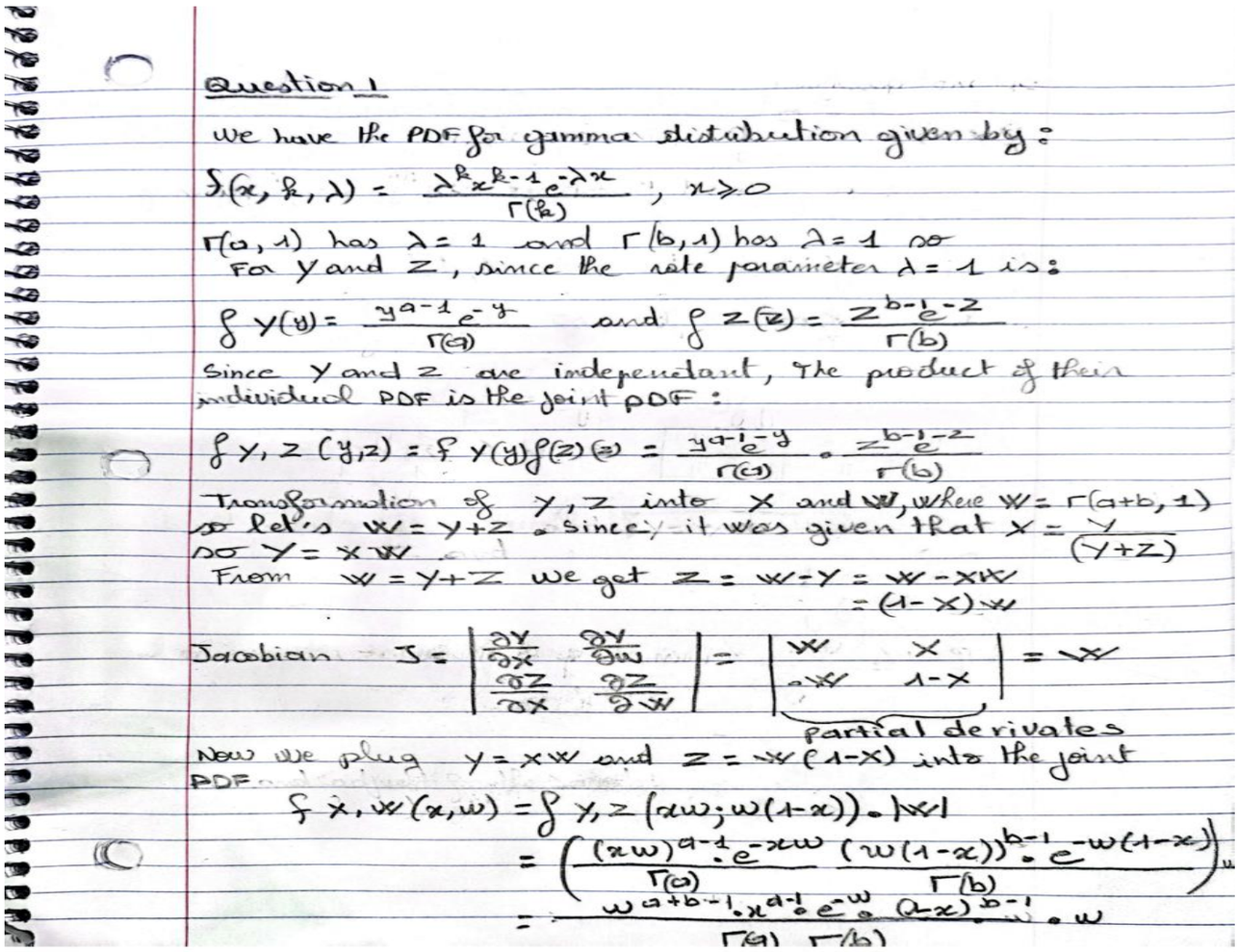
Yann-emmanuel Obrou

101365293

2025-01-29

Compulsory Questions

• Prove the following: if Y ∼ Γ(a, 1), Z ∼ Γ(b, 1), and Y, Z are independent, then X := Y /(Y + Z) ∼ Beta(a, b).



Question 1

We have the PDF for gamma distribution given by:

$f(x, k, \lambda) = \frac{\lambda^k x^{k-1} e^{-\lambda x}}{\Gamma(k)}$, $x \geq 0$

$\Gamma(a, 1)$ has $\lambda = 1$ and $\Gamma(b, 1)$ has $\lambda = 1$ so

For Y and Z, since the rate parameter $\lambda = 1$ is:

$f_Y(y) = \frac{y^{a-1} e^{-y}}{\Gamma(a)}$ and $f_Z(z) = \frac{z^{b-1} e^{-z}}{\Gamma(b)}$

Since Y and Z are independant, the product of their individual PDF is the joint PDF:

$f_{Y,Z}(y,z) = f_Y(y) f_Z(z) = \frac{y^{a-1} e^{-y}}{\Gamma(a)} \cdot \frac{z^{b-1} e^{-z}}{\Gamma(b)}$

Transformation of Y, Z into X and W, where $W = \Gamma(a+b, 1)$ so let's $W = Y+Z$ since it was given that $X = \frac{Y}{(Y+Z)}$ so $Y = XW$

From $W = Y+Z$ we get $Z = W - Y = W - XW = (1-X)W$

Jacobian $J = \begin{vmatrix} \frac{\partial Y}{\partial X} & \frac{\partial Y}{\partial W} \\ \frac{\partial Z}{\partial X} & \frac{\partial Z}{\partial W} \end{vmatrix} = \begin{vmatrix} W & X \\ -W & 1-X \end{vmatrix} = W$

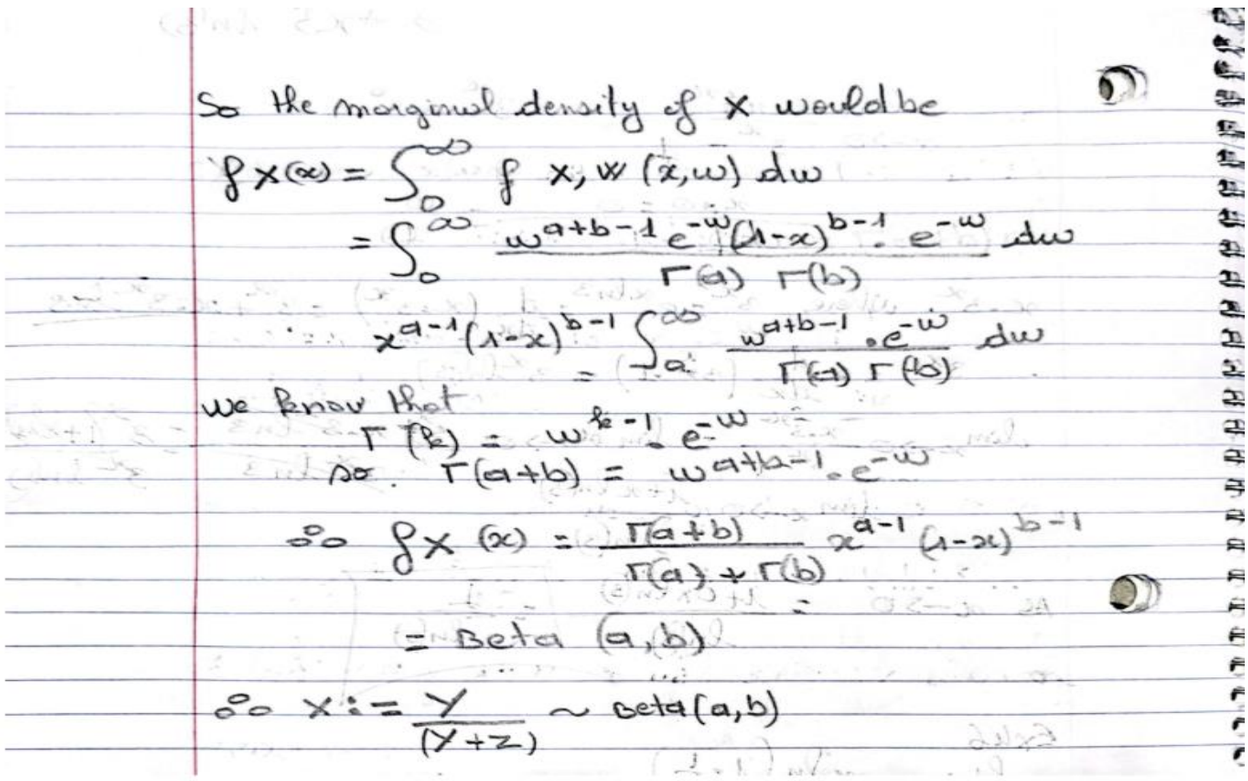
partial derivates

Now we plug $Y = XW$ and $Z = W(1-X)$ into the joint PDF.

$f_{X,W}(x,w) = f_{Y,Z}(xw; w(1-x)) \cdot |W|$

$= \left(\frac{(xw)^{a-1} \cdot e^{-xw}}{\Gamma(a)} \frac{(w(1-x))^{b-1} \cdot e^{-w(1-x)}}{\Gamma(b)} \right) w$

$= \frac{w^{a+b-1} \cdot x^{a-1} \cdot e^{-w} \cdot (1-x)^{b-1}}{\Gamma(a)\ \Gamma(b)} \cdot w$



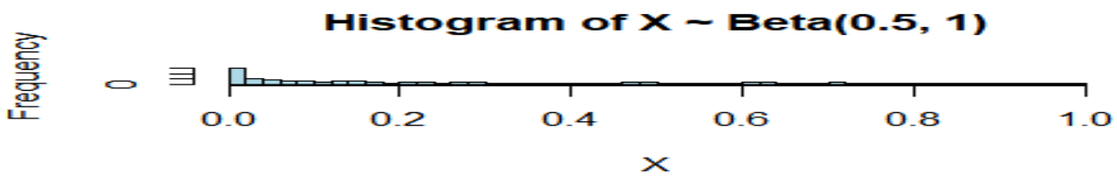
• Simulate a sample of size n = 1000 from X ~ Beta(1/2, 1).

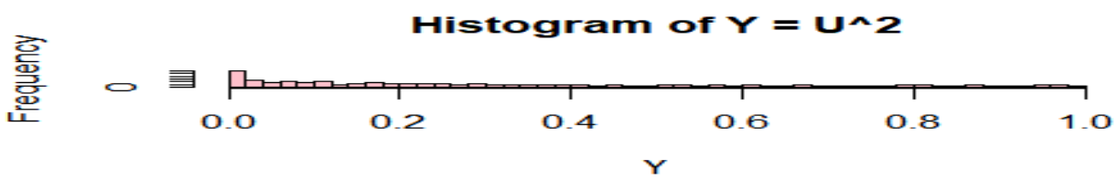
```
#set the parameters
n <- 1000
a <- 0.5
b <- 1
# Simulate Y ~ Gamma(a, 1)
Y <- rgamma(n, shape = a, rate = 1)
# Simulate Z ~ Gamma(b, 1)
Z <- rgamma(n, shape = b, rate = 1)
# Calculate X = Y / (Y + Z)
X <- Y / (Y + Z)
# Simulate X ~ Beta(0.5, 1)
X <- rbeta(n, shape1 = 0.5, shape2 = 1)
```

• Then simulate a sample of size 1000 by taking Y = U^2, with U being standard uniform.

```
#set the parameter and simulate
n<- 1000
U<- runif(n, 0,1)
Y<- U^2

# Plot histograms for X and Y
par(mfrow=c(2, 1))  # Set up the plot area for two plots
hist(X, breaks = 40, main = "Histogram of X ~ Beta(0.5, 1)", xlab = "X", col = "lightblue")
hist(Y, breaks = 40, main = "Histogram of Y = U^2", xlab = "Y", col = "pink")
```

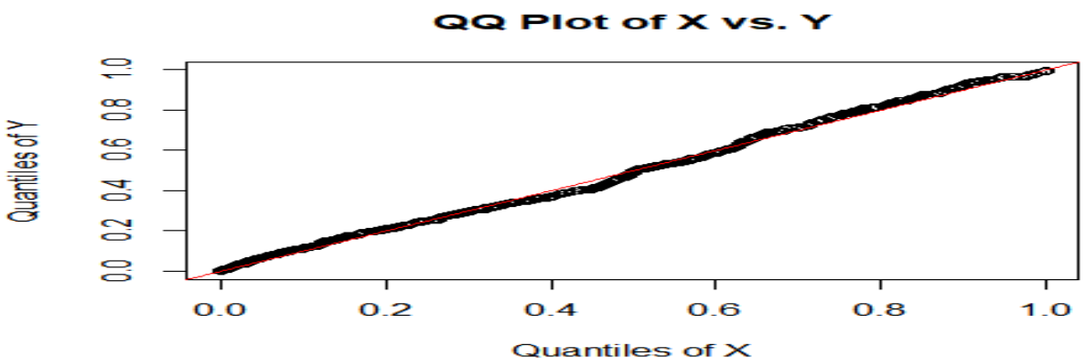




Both histogram for U^2 AND BETA(1/2,1) are the same

```
# Plot a QQ plot for X and Y
par(mfrow=c(1, 1))

qqplot(X, Y, main = "QQ Plot of X vs. Y", xlab = "Quantiles of X", ylab = "Quantiles of Y")
abline(0, 1, col = "red")
```



The qq plot show that the distribution follow the line y=x. Thus, the transformation U^2 of a uniform variable U results in a distribution that is mathematically equivalent to a Beta (1/2,1) distribution.

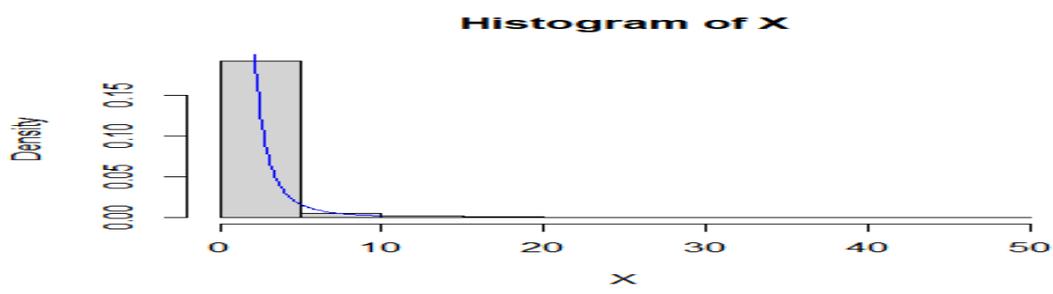
Question 2

• Use the inverse transform method to generate a sample of size n = 1000 from Pareto with α = 2

```
# Set parameters
alpha <- 2
n <- 1000
# Generate n uniform random variables in the range (0, 1)
u <- runif(n, 0,1)
# Apply the inverse transform method for Pareto distribution
X<- (1 - u)^(-1 / alpha)
# Display the first 10 samples
```

• Draw a histogram with the associated density.

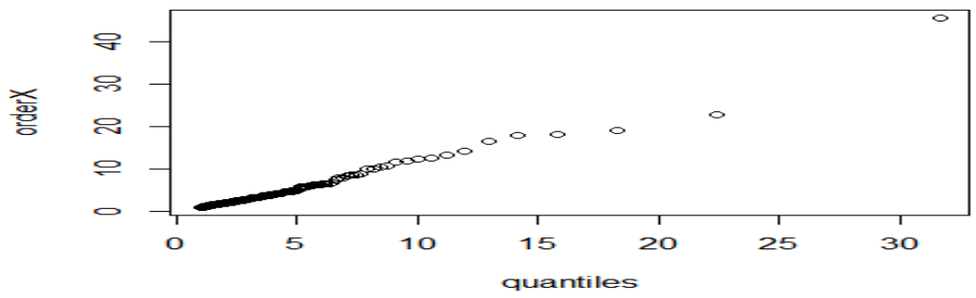
```
#create histogram
hist(X,freq=FALSE);
curve(alpha * x^(-alpha - 1),xlim=c(0,10),add=T,col="blue")
```



The histogram follows the expected theoretical density curve

• Create a QQ-plot.

```
#create a qqplot
orderX=sort(X); n=length(X);
grid=c(1:n)/(n+1);
quantiles=(1 - grid)^(-1 / alpha);
plot(quantiles,orderX)
```



• State a conclusion: do you data come from Pareto(α)? The histogram overlay with the theoretical Pareto density curve and the close alignment of histogram bars with the curve confirm that the simulated data closely match the expected long-tail behavior of the Pareto distribution. The QQ-plot further validates this by showing that the data points closely follow the line y=x, indicating that the quantiles of the simulated data align with those expected from a theoretical Pareto distribution. Therefore, the data generated through the inverse transform method accurately represent a Pareto distribution with α=2.

Question 3 (Generation of Gamma with non-integer shape, 4 points) Generate a sample of size 1000 from X ∼ Gamma(3/2 , 1).

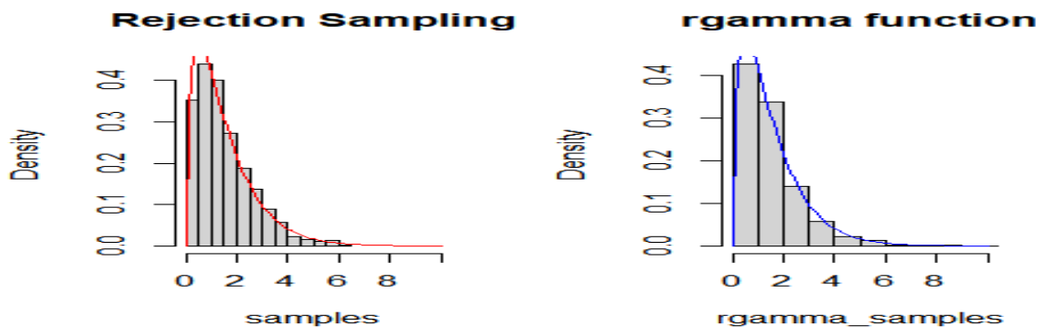
Use rejection method with g being exponential density with the same mean as X.

```
# Set parameters
n <- 1000  # sample size
alpha <- 1.5  # alpha, manually adjusted for a high acceptance rate
# Gamma PDF (target distribution)
f <- function(x) {
  (x^(3/2 - 1) * exp(-x)) / gamma(3/2)}
# Exponential PDF (proposal distribution)
lambda <- 2/3
g <- function(x) {
  lambda * exp(-lambda * x)}
# Rejection sampling
samples <- numeric(n)
count <- 0
while (count < n) {
  Y <- rexp(1, rate = lambda)  # Sample from g
  U <- runif(1)  # Uniform random number
```

```
  if (U <= f(Y) / (alpha * g(Y))) {
    count <- count + 1
    samples[count] <- Y}}
```

Plot relative frequency histogram and corresponding density on the same picture. Compare with rgamma function in R

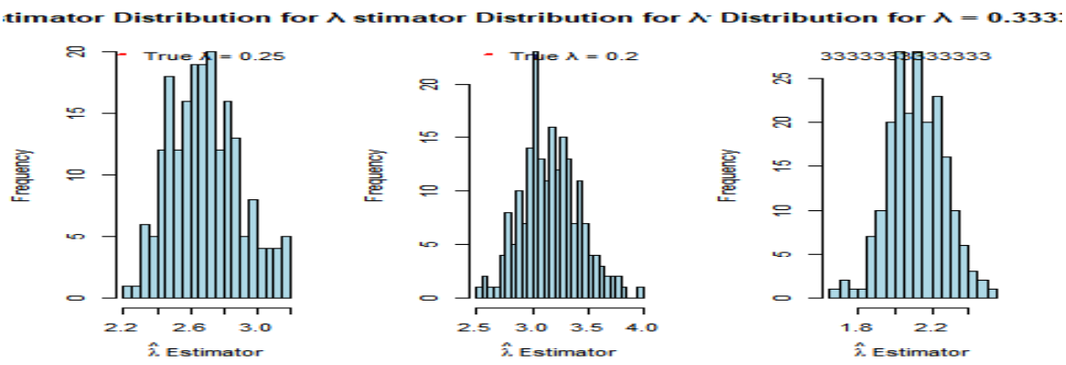
```
# Using rgamma for comparison
rgamma_samples <- rgamma(n, shape = 3/2, rate = 1)
# Plot results
par(mfrow = c(1, 2))
hist(samples, prob = TRUE, main = "Rejection Sampling", xlim = c(0, 10))
curve(f, add = TRUE, col = "red")
hist(rgamma_samples, prob = TRUE, main = "rgamma function", xlim = c(0, 10))
curve(f, add = TRUE, col = "blue")
```



Question 4

```
# Set parameters
set.seed(123) # For reproducibility
lambda_values <- c(1/4, 1/5, 1/3)  # Different true lambda values
n <- 200  # Number of laptops in the study
num_simulations <- 200  # Number of simulations per lambda

# Function to simulate the experiment
simulate_experiment <- function(lambda, n, time_limit, num_simulations=1000) {
  estimators <- numeric(num_simulations)
  for (sim in 1:num_simulations) {
    # Generate exponential lifetimes
    u <- runif(n)
    EXPO <- -log(u) / lambda
    # Create an indicator variable for laptops that fail within the 4-year limit
    B <- ifelse(EXPO <= time_limit, 1, 0)
    # Estimate lambda using only observed failures
    observed_failures <- EXPO[B == 1]
    lambda_hat_prime <- mean(observed_failures) / mean(B)  # Proposed estimator
     estimators[sim] <- lambda_hat_prime}
  return(estimators)}
# Run the experiment for different lambda values
results <- lapply(lambda_values, function(l) simulate_experiment(l, n, time_limit = 4,
num_simulations))
# Visualize the distribution of the estimator
par(mfrow=c(1, length(lambda_values)))
for (i in 1:length(lambda_values)) {
  hist(results[[i]], breaks=30, main=paste("Estimator Distribution for λ =", lambda_values[i]),
       xlab=expression(hat(lambda) ~ "Estimator"), col="lightblue", border="black")
  abline(v=lambda_values[i], col="red", lwd=2, lty=2)  # True lambda line
  legend("topright", legend=paste("True λ =", lambda_values[i]), col="red", lwd=2, lty=2,
bty="n")}
```




```
# Compute bias for each lambda
biases <- sapply(results, function(est) mean(est) - mean(lambda_values))
bias_table <- data.frame(Lambda=lambda_values, Bias=biases)
print(bias_table)

##      Lambda     Bias
## 1 0.2500000 2.428617
## 2 0.2000000 2.899972
## 3 0.3333333 1.857672
```

Is the proposed estimator unbiased? No, the proposed estimator lambda is biased: For Lambda=0.25, the bias is 2.408,For Lambda=0.20, the bias is 2.954. For Lambda =0.33, the bias is 1.807. This bias occurs because the estimator only considers the observed lifetimes within the 4-year limit, ignoring longer-lasting laptops. This underestimates the true average lifetime.

How does the bias behave for different values of lambda? The bias decreases as lambda increases. This trend suggests that when laptops last longer on average, more observations get cut off at 4 years, increasing bias.

Can we fix the procedure? Yes, we can reduce bias by: Following participants for more than 4 years to capture longer lifetimes.

Optional Question 1

Simulate a large sample from the Wigner law.

```
# Set parameters
n <- 1000  # sample size
c <- 1 / pi
# Define the Wigner PDF
f <- function(x) {
  (1 / (2 * pi) * sqrt(1 - x^2)) * (abs(x) <= 1)}
# Proposal distribution is uniform over [-1, 1], g(x) = 1/2
g <- function(x) {
  0.5 * (abs(x) <= 1)}
# Rejection sampling algorithm
samples1 <- numeric(n)
count <- 0
while (count < n) {
  Y <- runif(1, -1, 1)  # Sample from g, uniform over [-1, 1]
  U <- runif(1)
  if (U <= f(Y) / (c * g(Y))) {
    count <- count + 1
    samples1[count] <- Y}}
```

Draw a histogram.

```
hist(samples1, breaks = 40, probability = TRUE, main = "Histogram of Wigner Law Samples", xlab
= "x")
```

```r
# Add theoretical density for comparison, ensure curve is only plotted for valid x values
curve(f, from = -1, to = 1, add = TRUE, col = "red", lwd = 2)
```

